NEIL GANDHI | TECHNICAL WRITEUP | PERSONAL PROJECT

# Stock Dashboard

A self-updating stock dashboard that allows the analysis of some of the strongest companies in the world. Allows fundamental and earnings based analysis on continued growth metrics and parameters all with no server and no database.

Live Site: neilg.ca

## 1. Executive Summary

Most stock pages answer the easiest question in markets: what is the price right now? I built this dashboard to answer the one I actually invest on – what is the fundamental value of a company's operating business? A trailing price-to-earnings (P/E) ratio divides price by net income, and net income includes things that have nothing to do with the core business. Equity gains are the clearest case in my universe: they can make trailing P/E look low for Alphabet and Amazon in a quarter when the underlying business got no cheaper at all.

So the dashboard's primary multiple is one I define and compute myself, directly from SEC filings:

**Core operational P/E = Enterprise Value / TTM Operating Income**

Enterprise Value = Market Cap + Total Debt – Cash – Short-Term Investments. TTM = trailing twelve months. Every fundamental input comes from SEC EDGAR filings, not from a quote site's summary page.

The public dashboard lives in the Stocks view of my personal site, neilg.ca. It tracks nine stocks – Alphabet (GOOG), Meta (META), Microsoft (MSFT), Amazon (AMZN), Netflix (NFLX), Uber (UBER), Mastercard (MA), Visa (V) and ASML – with SPY as the sole benchmark. They are dominant technology, payments and semiconductor-equipment companies, picked for durable moats, strong margins and consistent revenue growth. The site has six tabs: a stock chat that answers only from the dashboard's own data, Valuation, Fundamental Analysis, Graph Analysis, Earnings + Filings, and a Simulation tab that back-tests a weighted mix of any tickers against a benchmark over 1, 2, 5 and 10 years.

What makes it a technical project rather than a spreadsheet is that nobody maintains it by hand. A Python job runs on a schedule, reads new filings from EDGAR, recomputes the valuation only when a filing actually changes it, and publishes the result as static JSON files that the page reads live. If the job fails, the page keeps showing the last good data instead of breaking. This writeup is written for engineers and hiring managers evaluating the project: what it does, how the data flows, the three problems that were genuinely hard, how to run the pipeline yourself, and where it still falls short.

**The short version:**

filings in, JSON out, static page on top – with the valuation recomputed from source documents every time a new 10-Q or 10-K lands.

## 2. Architecture & System Design

There is no application server behind this dashboard, and that is a deliberate choice, not a missing piece. The system is a data pipeline that ends in static files, plus a page that reads them. Hosting stays a plain GitHub Pages

site, every number on screen can be traced to a value in a JSON file anyone can open, and a failed refresh degrades to stale-but-labelled data instead of an error page.

*GitHub Action – weekdays 22:05 UTC (17:05 EST) – commits only when a file changes*




*Prices: Stooq, then Nasdaq, then Yahoo (first success wins). Two Cloudflare Workers sit beside the page: chat and Simulation history.*

## How a run flows

1. **The Action wakes up.** Weekdays at 22:05 UTC, after the North American close, GitHub Actions checks out the site repository and runs update-stocks-data.py on Python 3.12.
2. **Prices refresh every run.** Quotes come from a cascade – Stooq first, then the Nasdaq quote API, then Yahoo – so one source being down does not stall the run. That day's close is appended to each stock's price history and the 50- and 200-day simple moving averages (SMAs) are recomputed.
3. **Fundamentals move only on a filing.** The script checks EDGAR's submissions feed for each company. Only if the newest 10-Q, 10-K or 20-F has an accession number it has not seen does it recompute core P/E inputs and append a new quarter to the fundamental series. A price move alone never rewrites a valuation built on filed earnings.
4. **The page reads live.** The site fetches each JSON with a cache-buster on open. If a fetch fails, it falls back to a snapshot baked into the page, dated October 30, 2026 – labelled, dated data rather than a blank screen.

## Why this stack

Python, because the EDGAR work is JSON parsing and date arithmetic, and because a single script with no framework is easy to run identically on my machine and in CI. Static JSON as the data layer, because the read pattern is "nine stocks, read by everyone, written once a day" – a database would add cost and failure modes without buying anything. GitHub Actions as the scheduler, because the repository is already on GitHub and the commit history doubles as an audit log of every data change. Yahoo is used only where EDGAR cannot help – live prices and forward P/E – because its fundamentals endpoints return empty operating income fields for these companies, which is exactly the number this project exists to get right.

## The data layer is four files, each owned by one view

| File | Feeds | Updates when |
|---|---|---|
| **stocks-data.json** | Valuation: price, core / forward / average P/E, revenue growth, market cap, TTM operating income | Prices every run; core inputs only on a new filing |
| **stocks-graph.json** | Graph Analysis: daily closes, 50- and 200-day SMAs, signal | Each run appends that day's close, recomputes SMAs |
| **stocks-fundamental.json** | Fundamental: quarterly TTM revenue, EPS, FCF and EBITA, plus CAGRs | New quarter appended only on a new filing |
| **stocks-earnings.json** | Earnings + Filings: next earnings dates, SEC filings list | Filings appended from EDGAR; calendar best-effort |

A fifth file, stocks-earnings-summaries.json, holds the chat's short summaries of each stock's most recent earnings (beat or miss, revenue and EPS against consensus, guidance, what moved the stock). It is updated by hand each quarter – judging what moved a stock is not a scrape – and the automated job never touches it.

## Tech stack

| Layer | Choice |
|---|---|
| **Pipeline** | Python 3.12, standard library only – no dependencies to install |

| | |
|---|---|
| **Fundamentals** | SEC EDGAR companyfacts and submissions APIs |
| **Prices / forward P/E** | Stooq CSV, Nasdaq quote API, Yahoo quote API, in that order |
| **Scheduling / hosting** | GitHub Actions cron (weekdays 22:05 UTC, commits only on change); GitHub Pages, repo Quert-Z.github.io, domain neilg.ca |
| **Frontend** | One self-contained index.html (HTML, CSS, vanilla JavaScript) in a phone frame; no build step |
| **Edge services** | Two Cloudflare Workers: chat backend, and stock-sim-history for Simulation history |

### Two small Workers, two narrow jobs

The chat Worker answers questions from the dashboard's data, its system prompt keeping it strictly to the nine stocks and the dashboard itself; two fixed portfolio questions are answered on-device with canned replies instead of a model call. The history Worker exists because a static page cannot reliably fetch ten years of prices cross-origin: it takes one parameter (GET /?ticker=AAPL), returns weekly dates and closes as JSON, tries Stooq first and Yahoo second, and caches responses at the edge for a day. No keys, no storage, no state.

On open, a welcome tour introduces the six tabs, one short screen each, skippable throughout. Graph Analysis draws one chart per stock rather than combining them, so each company is read on its own scale.

# 3. Core Features & Implementation

### Challenge 1: a clean twelve months out of messy filing data

EDGAR's companyfacts feed does not hand you a trailing-twelve-month operating income. It hands you every value a company has ever tagged, in overlapping forms: discrete quarters, year-to-date figures, and annual totals, sometimes all three for the same period. Pick the wrong one and you double-count. The build follows three rules. First, always prefer the discrete quarterly value of OperatingIncomeLoss – the entry whose start and end dates span exactly one quarter. Second, derive the fourth quarter, because US filers never report Q4 on its own: it is the annual 10-K figure minus Q1, Q2 and Q3 from the 10-Qs. Third, respect fiscal calendars instead of forcing everyone onto December year-ends – Microsoft's year ends June 30 and Visa's ends September 30, so their quarter windows are computed from their own calendars.

```
def ttm_op_income(facts, report_date, fiscal):
    units = units_of(facts, "OperatingIncomeLoss")
    total = 0
    for (s, e, yr, q) in quarter_defs(report_date, fiscal):
        v = discrete_quarterly(units, s, e, yr, q, fiscal)
        if v is None:
            return None     # missing quarter: hold, do not guess
        total += v
    return total

# inside discrete_quarterly(), when no discrete Q4 exists:
#   Q4 = FY annual (10-K) - Q1 - Q2 - Q3 (the three 10-Qs)
```

Enterprise value is assembled to match: cash, short-term investments and debt are read from the balance sheet of the same filed quarter, market cap is the current price times shares outstanding, and the two are never mixed across periods. Every published row carries its filing vintage and price date, so a reader can tell which quarter produced the number.

**The ASML currency bug is the honest example of why this matters.** ASML reports in euros but its US listing is priced in dollars. The first version divided a dollar enterprise value by a euro operating income and printed a core P/E of 39.1x – precise-looking and wrong. Converting TTM operating income of EUR 15.22 billion at the recorded

rate of 1.1234 EUR/USD gives $17.10 billion and a core P/E of 34.8x. The rate is stored on the row itself so the conversion can be audited later. ASML is also a foreign filer (20-F and 6-K, no 10-Q), which is why its row is carried forward rather than appended by the script – more on that in Section 5.

The pipeline was verified end to end against the filings: Mastercard's TTM operating income for Q3 2026 recomputed from companyfacts came out at $21.266 billion, matching the published JSON exactly, and Visa's TTM revenue and operating income matched its FY2026 10-K filed November 5, 2026.

## The pipeline has to be safe to leave running

## Challenge 2: a job that rewrites its own data, unattended

An automated job that edits the files a live page reads can do real damage quietly, so most of the engineering went into guards, not features. The recompute gate is the main one: valuation inputs are rebuilt only when EDGAR shows a filing whose accession number differs from the one already stored, so the filing-dependent work happens at most once per quarter per company.

```
needs_recompute = (prev.get("ttm_op_income") is None) or \
    (latest and latest[2] != prev.get("last_acc"))
```

Everything else follows the same principle – carry forward what you cannot freshly verify, and never write a null over a good value:

- **Filings guard.** Only filings newer than the newest date already in the file are appended, capped at 400. The guard exists because an early unguarded run swallowed 4,769 old filings in one pass.
- **History guard.** A date already in the price history is never appended twice; history is capped at 1,500 entries.
- **Price cascade.** Stooq, then Nasdaq, then Yahoo. If all three fail, prices carry forward instead of crashing – and as_of is derived from actual price dates, so it stays honest about freshness.
- **Identifier check.** Visa's CIK was corrected to 0001403161 after an earlier value turned out to be a different company. A wrong CIK silently fetches someone else's filings.
- **Per-stock isolation.** Each stock refreshes in its own try block (one-second pause between EDGAR calls), so one failure cannot take the other eight down.

## Challenge 3: ten years of history for any ticker, from a static page

The Simulation tab builds a portfolio from any tickers, sets each weight with a slider, and plots its growth against a benchmark (SPY by default) over 1, 2, 5 or 10 years. Both lines start at $100, buy-and-hold, no rebalancing: the portfolio line is the weight-blended growth of its holdings, each rebased to the window's first close. The problem is data – the dashboard's files only ever held about three years, and a browser page cannot reliably pull a decade of prices cross-origin.

The fix splits in two. For SPY and the nine tracked stocks, weekly history back to June 2015 (dividend-adjusted closes) is baked into the page, so every window including 10Y draws with no fetch at all; Uber still starts at its 2019 IPO. For any other ticker, the page asks the stock-sim-history Worker, which fetches full history server-side, reduces it to weekly closes, and falls back from Stooq to Yahoo when the first source blocks or returns too little:

```
try {
  return json(await fromStooq(ticker));
} catch (e1) {
  try { return json(await fromYahoo(ticker)); }
  catch (e2) { return json({ error: "no history found" }, 502); }
}
```

In practice Yahoo does most of the work – Stooq blocks the Worker's requests, so the fallback is the common path, and its adjusted-close series is better data anyway. If the Worker is unreachable, the tab tries public CORS proxies; failing that, tracked tickers still work from the baked series.

# 4. Setup & Usage Guide

## Prerequisites

- Python 3.12 – standard library only, nothing to pip install.
- A GitHub repository with Pages enabled, holding index.html, the updater, the workflow file, and the four seed JSONs.
- A free Cloudflare account for the chat and history Workers; the core tabs work without one.
- No API keys and no .env file. EDGAR asks for a descriptive User-Agent header; it is a constant in the script, not a secret.

## Run the pipeline locally

```
git clone https://github.com/Quert-Z/Quert-Z.github.io.git
cd Quert-Z.github.io
python3 update-stocks-data.py
```

The script reads the existing JSON files in its own directory first, then writes updated versions of all four. Two behaviours matter. First, the seed files must exist: if the graph, fundamental or earnings file is missing, that part is skipped with a log line – the script appends to history, it does not backfill it, so a deleted file is restored from the repository, not regenerated. Second, a run with no new filings still updates prices, graph closes and moving averages, while valuation and fundamental data carry forward untouched.

## Turn on the automation

The workflow file (stocks-data.yml) is the whole scheduler:

```
on:
  schedule:
    - cron: '5 22 * * 1-5'    # 22:05 UTC, Mon-Fri
  workflow_dispatch:        # manual runs from the Actions tab
permissions:
  contents: write           # so the job can commit the JSON
```

After the script runs, the job stages the four JSON files and commits only if something changed. To deploy the history Worker, create a Worker named stock-sim-history in the Cloudflare dashboard, paste in the Worker code, and deploy; then set SIM_WORKER_URL in index.html to its workers.dev URL. Test it first: …workers.dev/?ticker=AAPL should return JSON whose dates and closes arrays reach back years. One note from experience: an index.html taken from the site's Deploy-tab download must go through the portable patch script before pushing (the download bundles a runtime shim that hangs on GitHub Pages), and Pages needs roughly five minutes to deploy before you check.

## Using the dashboard

Open neilg.ca and enter the Stocks view. Ask the chat about any of the nine stocks – price against its moving averages, revenue growth, forward P/E, latest TTM totals, next earnings date, or how the last quarter went. In Valuation, tap a sort pill to reorder by that multiple. In Simulation, add tickers (non-US listings need their exchange suffix, e.g. VFV.TO for Toronto), drag the weight sliders or tap "Balance weights evenly", pick a window, and read the portfolio line against the benchmark.

# 5. Limitations & Future Roadmap

## What is missing, stated plainly

- **Earnings dates are the weakest data in the system.** No free source proved reliable: one API returns empty data, another only covers near-term dates, and Yahoo's calendar endpoint rate-limits. So a stored date is replaced only by a confirmed future date, a passed date is marked "reported", and dates are never invented. The cost is real: the calendar may go stale after January 2027.
- **ASML is maintained by hand.** As an IFRS foreign filer it is carried forward whole – its valuation row uses a euro conversion at a fixed recorded rate, and its fundamental quarters are not appended automatically.
- **Some gaps are in the source, not the code.** Visa has no diluted EPS or share-count tags in companyfacts, so its EPS is null. Mastercard and Visa tag amortization only annually, so no quarterly EBITA can be derived for them; Alphabet and Meta disclose it in filing notes, so their new quarters stay null while history carries forward. These render as absent, never as zero.
- **Coverage is asymmetric.** The filings list is appended for six of the nine stocks (GOOG, META, AMZN, MSFT, NFLX, UBER); Mastercard, Visa and ASML filings are not added by the script.
- **Freshness is bounded by the schedule.** Data is only as new as the last successful weekday run, the fallback snapshot is frozen at October 30, 2026 if live fetches fail, and the QQQ forward P/E the chat compares against (21.6x) is a stored constant, refreshed by hand.
- **The Simulation is only as uniform as its sources.** Tracked tickers use the baked dividend-adjusted weekly series; tickers fetched through the Worker use whichever series the source returns – Yahoo's adjusted close when available, Stooq's split-adjusted close otherwise.

## Roadmap, in the order I would do it

1. **A reliable earnings-date source.** The current best-effort refresh is the one part of the pipeline I do not fully trust, and it is the first thing to fix – either a dependable feed, or deriving dates from the filings and releases themselves as they land.
2. **Automatic ASML appends.** Parse the 20-F and interim 6-K reports so ASML's fundamentals update like the US filers', with the euro conversion done at the rate current at each filing instead of a stored one.
3. **Filings coverage for the remaining three.** Extend the append path to Mastercard, Visa and ASML so the Earnings + Filings tab covers the full universe the dashboard tracks.
4. **A freshness check on the stored constants.** The QQQ comparison multiple and the baked fallback snapshot should flag themselves when they age past a set window, instead of relying on me remembering.

None of that changes the shape of the system, and that is the point. The design bet – filings as the source of truth, a scheduled job instead of a server, static files a page can read live – has held up through every fix above. The remaining work is widening the parts that are still manual.

*Method and sources: SEC EDGAR companyfacts and submissions feeds; dashboard data schema and verification notes dated November 5–6, 2026, including the Mastercard TTM operating income spot check ($21.266 billion, Q3 2026) and the ASML core P/E correction (39.1x to 34.8x at 1.1234 EUR/USD). Prices and multiples are as of the price date recorded on each row. This document describes a personal project and is not investment advice.*